

2026 C语言

帖子: https://blog.csdn.net/weixin_44617968/article/details/117661182

1. sizeof运算符

用来计算一个变量或常量、数据类型所占的内存字节数

(如果计算数据类型, 则不能省略括号)

1. sizeof常用

Char=1字节 (8bit), short=2, int=4, float=4, double=8

指针 (32位机器=4, 64位=8), long (32位机器=4, 64位=8)

结构体: 内存对齐后所占内存字节数

表达式: 返回表达式结果的类型所占内存字节数, 不会运行表达式

2. sizeof特例

(1) sizeof (数组名): 计算的是整个数组的所占内存字节数 (这里的数组名表示整个数组)

数组名: 一般表示数组首元素的地址

&数组名: 这里的数组名表示整个数组, 取出的是整个数组的地址

(2) sizeof (arr[0]): 计算的是数组第一个元素的所占内存字节数

(3) sizeof (字符串): **sizeof** 在计算大小的时候也会将 `'\0'` 计算在里面
给字符串赋值的时候, 字符串末尾自动补上了一个 `'\0'`

字符串是字符数组, 但最后以`\0`结尾

3. strlen (字符串/字符数组): 返回`'\0'`之前的字符个数, 直到找到`'\0'`才停止

4. sizeof和strlen的区别

(1) sizeof是编译期运算符 (变长数组vla是在运行时执行的)

strlen是字符串函数, 在程序运行时执行

(2) sizeof 计算的是变量或常量、变量类型所占的字节数

strlen计算的是`'\0'`之前的字符个数 (遇到`'\0'`才停止), 只能用char*或者const char*做参数

strlen计算时不包含`'\0'`, sizeof计算时包含`'\0'`

2. 逗号运算符

把多个表达式连接起来组成一个表达式（逗号表达式）

逗号运算符会从左至右依次取出每个表达式的值, **最后整个逗号表达式的值等于最后一个表达式的值**

3. 函数

1. 参数

- (1) 形参变量只有在被调用时才分配内存单元,在调用结束时,即刻释放所分配的内存单元
- (2) **当数组名作为函数参数时, 自动转换为了指针类型**, 在函数中无法动态计算数组的元素个数

4. CPU读写内存

- 通过地址总线找到存储单元的地址
- 通过控制总线发送内存读写指令
- 通过数据总线传输需要读写的数据

变量的存储原则

- 先分配字节地址大内存,然后分配字节地址小的内存(内存寻址是由大到小)
- 变量的首地址,是变量所占存储空间字节地址(最小的那个地址)
- 低位保存在低地址字节上,高位保存在高地址字节上（小端模式）

5. unsigned和signed

signed和unsigned的区别就是它们的最高位是否要当做符号位

6. 数组

一组相同类型数据的集合

1. 数组内部存储细节

存储方式:

- 1)内存寻址从大到小, **从高地址开辟**一块连续没有被使用的内存给数组
- 2)从分配的连续存储空间中, **地址小的位置开始给每个元素分配空间**
- 3)从每个元素分配的存储空间中, **地址最大的位置开始存储数据**
- 4)**用数组名指向整个存储空间最小的地址**

2. 二维数组

可以省略行数，但是不能省略列数

7. 指针

1. 指针变量：专门用于存储其它变量地址的变量

野指针：指向无效/非法内存地址的指针

二级指针：如果一个指针变量存放的又是另一个指针变量的地址,则称这个指针变量为指向指针的指针变量

2. 运算符

&变量名：表示变量的地址

*：在定义变量的时候 * 是一个类型说明符,说明定义的这个变量是一个指针变量

在不是定义变量的时候 * 是一个操作符,代表访问指针所指向存储空间

3. 指针和字符串

(1) 定义字符串

字符数组：使用字符数组来保存的字符串是保存栈里的,可以修改字符串中的字符

字符指针：使用字符指针来保存字符串,它保存的是字符串常量地址,常量区是只读的,不可以修改字符串中的字符

4. 指针和函数

(1) 指针函数：本质是函数，返回值是指针（内存地址）

语法上：返回类型* 函数名（参数）

常见用途：从 Flash/EEPROM 读取配置数据后，返回数据的首地址；或者动态申请内存（malloc）后返回内存指针，还有驱动开发中返回硬件寄存器的地址等。

(2) 函数指针：本质是指针，指向一个函数的内存地址

语法上：返回类型 (*指针名)(参数)

常见用途：中断服务函数的回调（外设触发中断后，通过函数指针调用对应的处理函数）、驱动中的事件回调（比如按键按下后调用不同的处理函数）、或者实现状态机（不同状态对应不同的处理函数，通过函数指针切换）

4. 野指针：指向无效/非法内存地址的指针

指针定义后未初始化、指针指向的内存被释放后未置空（悬空）、指针越界访问

(1) 与空指针区别：空指针可以判空检测，野指针不可控

(2) 避免野指针：“初始化 + 边界检查 + 规范管理”

指针定义时强制初始化

动态内存分配后必做判空

硬件寄存器指针：用宏封装，避免手动赋值错误

使用指针前必做有效性检查

释放内存后，指针立即置 NULL

避免指针复用混乱

(3) 嵌入式中特有的辅助手段

利用编译器 / 工具链检测：开启编译器警告

硬件层面的内存保护（MPU）：STM32 等 MCU 支持内存保护单元（MPU），可划分内存区域的访问权限，野指针访问非法区域时会触发硬件异常；

避免在中断中使用动态指针：中断服务函数（ISR）中禁止用 `malloc` / `free`，避免内存碎片导致指针指向非法地址，**中断中优先用静态缓冲区**

8. 结构体

保存一组不同类型的数据

1. 访问结构体变量

引用的一般形式为：`结构体变量名.成员名`

通过结构体指针访问结构体成员, 可以通过以下两种方式

`(*结构指针变量).成员名`

`结构指针变量->成员名`

2. 内存对齐问题

实际的计算机系统对基本类型数据在内存中存放的位置有限制, 它们会要求这些数据的起始地址的值是某个数k的倍数

一来简化了处理器与内存之间传输系统的设计, 二来可以提升读取数据的速度, 减少内存操作

3. 结构体占用存储空间计算

先按成员自身大小分配空间, 再遵循“内存对齐”规则

(1) 编译器默认, 结构体的大小是其**最大基本数据类型成员的整数倍** (常见的是按 4 字节 / 8 字节对齐, 也可通过 `#pragma pack` 调整)

(2) 结构体成员会**按定义顺序依次存储**, 但**每个成员的起始地址必须是其自身大小的整数倍**

(3) 特殊情况

数组按元素类型和个数计算

嵌套结构体: 先计算嵌套结构体的大小 (按自身对齐规则), 再作为一个整体参与外层结构体的对齐 (外层最大成员包含嵌套结构体的最大成员)

4. 为什么要内存对齐

因为嵌入式处理器访问非对齐的内存地址时, 可能会触发硬件异常, 或者访问效率大幅降低

实际开发中：如果追求内存紧凑，会调整结构体成员顺序减少空洞；如果追求访问效率，会遵循默认对齐；也可通过 `#pragma pack(1)` 强制按 1 字节对齐（取消空洞）

9. 共用体

1.和结构体不同的是, 结构体的每个成员都是占用一块独立的存储空间, 而共用体所有的成员都占用同一块存储空间

2.共用体的应用场景

(1) 通信中的数据包会用到共用体, 因为不知道对方会发送什么样的数据包过来, 用共用体的话就简单了, 定义几种格式的包, 收到包之后就可以根据包的格式取出数据。

(2) 节约内存。如果有2个很长的数据结构, 但不会同时使用, 比如一个表示老师, 一个表示学生, 要统计老师和学生的情况, 用结构体就比较浪费内存, 这时就可以考虑用共用体来设计。

(3) 某些应用需要大量的临时变量, 这些变量类型不同, 而且会随时更换。而你的堆栈空间有限, 不能同时分配那么多临时变量。这时可以使用共用体让这些变量共享同一个内存空间, 这些临时变量不用长期保存, 用完即丢, 和寄存器差不多, 不用维护

10. 枚举

1.枚举常量：C语言编译器会将枚举元素作为整型常量处理

枚举元素的值取决于定义时各枚举元素排列的先后顺序。默认情况下, 第一个枚举元素的值为0, 第二个为1, 依次顺序加1。

也可以在定义枚举类型时改变枚举元素的值

11. auto和register关键字

1.auto关键字

- 只能修饰局部变量, 局部变量如果没有其它修饰符, 默认就是auto的
- 特点: 随用随开, 用完即销

2.register关键字(忘记)

- 只能修饰局部变量, 原则上将内存中变量提升到CPU寄存器中存储, 这样访问速度会更快
- 但是由于CPU寄存器数量相当有限, 通常不同平台和编译器在优化阶段会自动转换为auto

12. static关键字

static 变量存储在**静态存储区**（而非栈 / 堆），单片机复位前值会一直保留，但复位后会初始化

1.修饰局部变量：**改变局部变量的存储位置，延长局部变量的生命周期，保留值不销毁**

普通局部变量存在栈区，函数执行完就销毁；用 static 修饰后，**变量会被放到全局 / 静态存储区**，程序运行期间只初始化一次，即使函数退出，值也会保留

应用场景：单片机的按键消抖函数，需要记录上一次的按键状态

2. 修饰全局变量 / 函数：**改变外部链接属性，限制作用域，仅在当前源文件可见，避免全局命名冲突**

普通全局变量 / 函数的作用域是整个工程（外部文件可通过 extern 引用）；用 static 修饰后，作用域被限制在定义的 `.c` 文件内，避免全局命名冲突。

应用场景：嵌入式开发中一个外设内部的辅助函数 / 全局变量会用 static 修饰，只在源文件内使用，**防止和其他驱动的同名函数冲突**

13. extern关键字

声明“外部符号”（变量 / 函数），告诉编译器该符号在其他源文件中定义，当前文件仅做引用

1. 声明外部全局变量

如果在 A 文件定义的全局变量，想在 B 文件中使用，用 extern 声明后，编译器就知道这个变量的定义在其他 `.c` 文件中，链接阶段会自动找到它

应用场景：嵌入式开发中，比如单片机的系统状态变量通常定义在 `main.c` 中，而驱动文件 `uart.c` 需要读取这个状态，就会用到 extern

2. 声明外部函数（简化头文件使用）

函数默认是“外部链接属性”，不用 extern 也能跨文件调用，但显式用 extern 声明（通常写在 `.h` 头文件中）是嵌入式开发的规范写法，能明确告诉编译器“这个函数的实现在其他 `.c` 文件中”。

应用场景：比如 I2C 驱动的 `i2c_write` 函数定义在 `i2c.c` 中，要让 `lcd.c` 调用这个函数，规范做法是在 `i2c.h` 中用 extern 声明

14. 编译过程

1. **预处理**：根据预处理指令，进行包含头文件，处理条件编译和展开宏定义，将.c文件预处理为.i文件

2. **编译**：将预处理后的代码转换为汇编代码的过程，会进行语法检查和语义分析，将.i文件编译为.s汇编文件

3. **汇编**：将汇编代码翻译成机器码的过程，将汇编指令翻译成相应的二进制指令，将汇编文件汇编为.o目标/二进制文件

目标文件里的代码还不能直接运行，因为函数 / 变量的地址是“临时的”，且没关联启动文件、库文件

4. **链接**：将多个目标文件和库文件链接成一个可执行程序的过程

将各个目标文件中引用的库函数或变量解析为实际地址

单片机：合并目标文件 + 库文件，生成可重定位的 ELF 文件

需要用 `objcopy` 工具把 ELF 文件转换成**纯二进制文件**（`.bin`）或 Intel Hex 文件

`.bin`：裸二进制，按地址顺序存储代码 / 数据，体积小；

`.hex`：ASCII 格式，包含地址信息，适合烧录工具识别

15. typedef关键字

1. 简化复杂类型的书写

嵌入式开发中会统一给基础类型起别名，以适配不同位数的 MCU，只需修改 typedef 的底层类型

`typedef unsigned char uint8_t;`

2. 简化结构体/枚举的使用

比如单片机的 GPIO 配置结构体

3. 定义函数指针别名

比如单片机的中断回调函数指针

16. typedef和define的区别

1. typedef在编译阶段处理，为已有类型创建“别名”，有类型检查

define在预处理阶段进行文本替换，无类型检查

2. typedef用来给类型起别名

常量 / 地址 / 宏开关用 `#define`

17. 带参宏定义和函数的区别

1. 带参宏定义在预处理阶段进行纯文本替换，无编译 / 类型检查

函数在编译阶段处理，生成独立代码块，有类型检查

2. 带参宏定义无调用开销，但会增加 Flash 占用

函数调用时需压栈 / 传参 / 跳转 / 返回，有调用开销，但减少Flash占用

3. 带参宏定义无显式返回值

函数有明确返回值类型

4. 带参宏定义预处理后代码被替换，无法单步调试宏内部逻辑

应用场景：

1. 带参宏定义

高频小逻辑（位操作 / 寄存器配置）

中断服务函数（ISR）内的逻辑

2.函数：

逻辑复杂 / 多行代码（如数据校验）

对类型安全要求高（如参数校验）

Flash 资源紧张（如 51 单片机）

18. const关键字

核心作用是定义“只读”的变量 / 数据，限制变量的修改权限，const总是先修饰其左侧标识符

1.修饰普通变量 / 数组，定义只读数据

编译器会禁止对其赋值修改；嵌入式中会把 const 修饰的全局数组 / 变量放到**Flash（程序存储区）**而非 RAM，节省单片机宝贵的 RAM 资源。

2.修饰指针

（1）**常量指针**：（const 类型 * 指针）指向“只读内容”的指针

指针本身可以指向不同地址，但指针指向的内存内容**不能被修改**

应用场景：单片机的**状态寄存器**（GPIO 的输入状态寄存器）是只读的，用常量指针保护，避免误写

（2）**指针常量**：（类型 * const 指针）“地址固定”的指针

指针变量本身的地址**不能修改**（只能指向初始化的地址），但指向的内存内容可以改

应用场景：单片机的**外设控制寄存器**（如 UART 的数据寄存器）地址固定，用指针常量保证指针不被误指向其他地址

（3）指向常量的常量指针：（const 类型 * const 指针）“既固定地址，又只读内容”的场景（比如只读的配置寄存器）

3.修饰局部变量：函数参数/返回值 / 结构体成员

修饰函数返回值时，禁止直接修改返回的只读数据；修饰结构体成员时，限制成员只读

4.const和#define的区别

- `#define`：预处理阶段文本替换，无类型检查，仅“常量”；
- `const`：编译阶段的只读变量，有类型检查，可占内存（存 Flash/RAM）；

嵌入式中：固定常量（如寄存器地址）用 `#define`，只读数据（如配置表）用 `const`

19. 内存管理

1.由C/C++编译的程序占用的内存

（1）代码段：存储程序的执行代码，即程序的机器指令

只读，代码段的大小在程序加载时被确定，通常是固定大小

(2) 数据段：包含程序中的全局变量、静态变量和常量

分为初始化数据段（.data段）和未初始化数据段（.bss段）

初始化数据段存储已经初始化非0的全局和静态变量，而未初始化数据段存储未初始化的全局和静态变量以及为0的

(3) 堆：程序运行时动态分配内存的区域，用于存储程序在运行时创建的数据

堆的大小不固定，在程序运行时可以动态增长或缩小

程序通过对应的内存管理函数（如malloc、free或new、delete）来进行对堆内存的分配和释放

(4) 栈：存储程序运行过程中的局部变量、函数参数、返回地址等数据

栈是一种后进先出（LIFO）结构，每次函数调用时会在栈上创建一个新的帧用于存储方法的参数和局部变量

栈的大小通常较小且固定，当递归层级过深或者栈溢出时会导致程序崩溃

(5) 全局/静态变量区：有些编译器会独立分出一个全局/静态变量区域存储全局变量和静态变量，这些变量在程序整个运行过程中都有效

2.malloc函数

申请堆内存空间并返回,所申请的空间并未初始化（memset 字节初始化）

void * 成功返回非空指针指向申请的空间，失败返回 NULL

3.free函数：释放申请的堆内存

4.calloc函数：申请堆内存空间并返回，所申请的空间，自动清零

5.realloc函数：扩容(缩小)原有内存的大小。通常用于扩容，缩小会会导致内存缩去的部分数据丢失

20. volatile关键字

核心作用：告诉编译器**变量可能被异步修改，禁止缓存优化，强制每次从内存读取最新值**

1.防止缓存优化：多次读取同一个变量时，会把变量值暂存到 **CPU 寄存器**，减少内存访问，提升效率。有些变量（如硬件寄存器、中断修改的变量）可能被 CPU 之外的硬件 / 中断异步修改，此时优化会导致读取到 “过期值”

volatile 就是禁止这种优化，**强制每次从内存（硬件 / 真实地址）读取**

应用场景：

1.修饰硬件寄存器地址

外设寄存器（如 UART 数据寄存器）的值会被硬件异步修改

2.修饰中断 / 多线程共享变量

中断服务函数和主程序共享的变量，或多任务（RTOS）共享的变量，会被异步修改

3.修饰被硬件 DMA 修改的缓冲区

DM会绕过 CPU 直接读写内存缓冲区，保证 CPU 读取到 DMA 最新写入的数据

21. void关键字

1.void *：任何类型的指针都可以直接赋值给它，无需进行强制类型转换

(void * 不可以无需强制类型转换地赋给其它类型的指针)

2.void关键字的使用规则

(1) 函数无返回值

凡不加返回值类型限定的函数，就会被编译器作为返回整型值处理

(2) 函数无参数

在 C++ 中，函数参数为 void 的意思是这个函数不接受任何参数

(3) 函数的参数是任意类型指针

任何类型的指针都可以传入 memcpy 和 memset 中

3.void (*b[10])(void (*)());的解说

(1) (*b[10])，其中出现了变量名 b，很容易就知道，b 是一个有 10个元素的数组，每个元素都是一个指针。

(2) (void(*)()), 其中没有出现变量名，所以它代表了一个类型，即“指向返回值为 void 型的函数的指针”

(3) C 语法中，类型修饰符是必须出现在变量名的左边，而在整个表达式中这个类型符是在变量名 b 的右边，所以 (void(*)()) 最外层的这个()，表示了定义了一个函数，这个函数有一个参数，就是一个指针，具体来说，就是“指向返回值为 void 型的函数的指针”

(4) b 数组里，每一个指针元素，都是一个函数指针，这个函数有一个参数，这个参数是一个函数指针。整个表达式最左边的 void，则定义了 b 数组中函数指针所指向函数的返回值类型。

22. 从堆和栈上建立对象哪个快

在堆上建立对象通常比在栈上建立对象慢

1.堆上的内存分配是动态的，需要系统在运行时查找合适的内存空间并进行分配，而且需要额外的内存管理开销。

在栈上建立对象是一种静态内存分配，编译器在编译时就可以确定内存分配的位置和大小，因此更加高效

2.在堆上建立对象时，程序需要调用new或malloc等动态内存分配函数，再将返回的指针赋给对象指针。

而在栈上建立对象时，只需要在声明对象时指定对象类型，编译器会自动分配内存并调用对象的构造函数。

23. #include<filename. h>和#include" filename. h"

- 1.对于 include<filename. h>, 编译器先从**标准库路径**开始搜索filename.h, 使得系统文件调用较快。
- 2.而对于# include"filename.h", 编译器先从**用户的工作路径**开始搜索 filename.h, 然后去寻找系统路径, 使得自定义文件较快。

24. 标准库函数

- 1.memcpy() (内存拷贝函数)

从源头指向的内存块拷贝固定字节数的数据到目标指向的内存块。

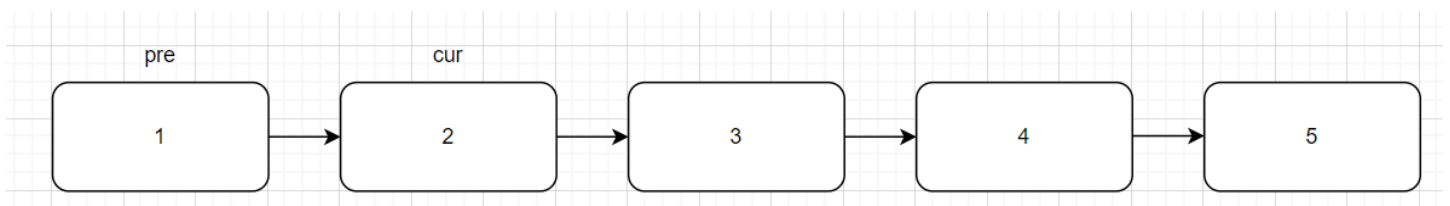
(与strcpy()函数不同的是,该函数不会检查任何终止字符(如'\0'),而总是精确的拷贝参数传入的字节数)

返回拷贝后的目的地内存块的起始地址

25. 链表翻转

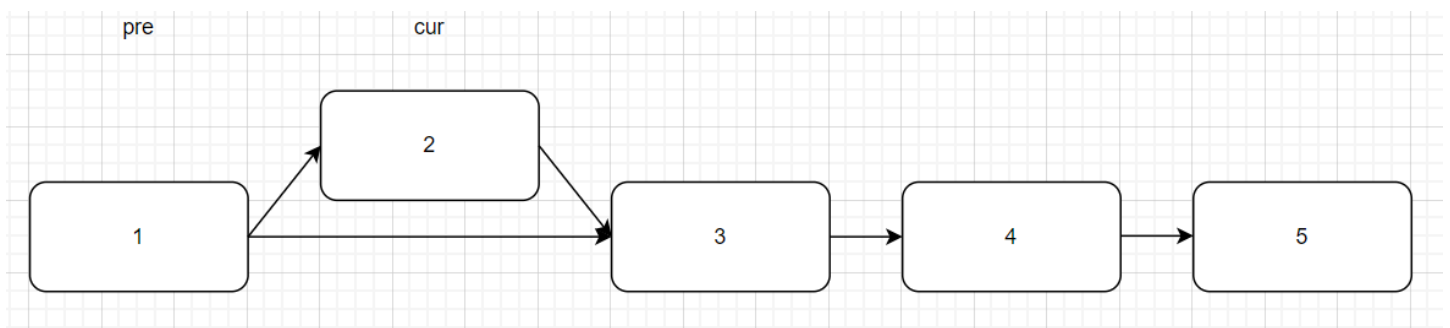
- 1.原地翻转 (不带头结点)

原地反转需要两个指针, 以防 `pre` 指针操作后指向上一节点而丢失指向下一节点的指针



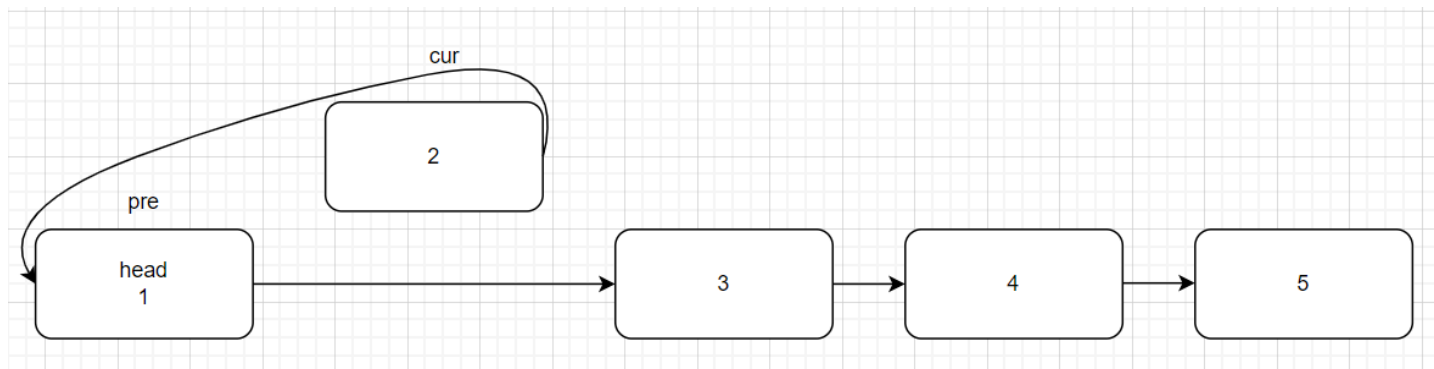
(1) 连: `pre->next = cur->next`

先将 `pre` 所在节点和 `cur->next` 所在节点连接起来,将当前节点 `cur` 从链表中移除, 并把链表连接起来



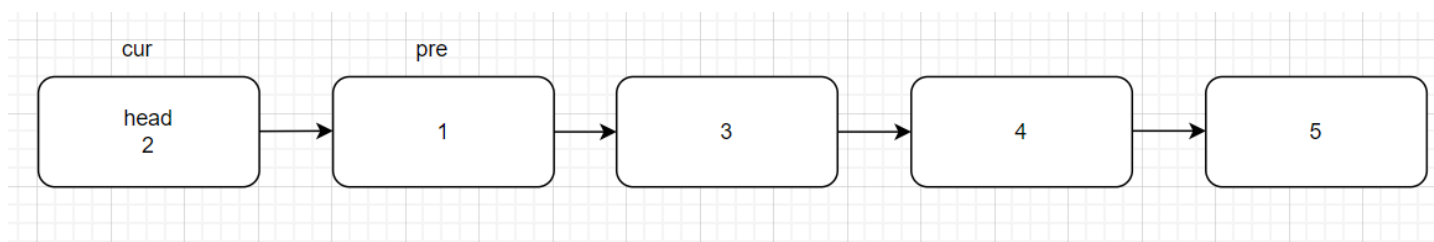
(2) 掉: `cur->next = head`

将cur的next指向head, 将cur插入到head的前面,以便将其放置到链表的前面, 成为新的头节点



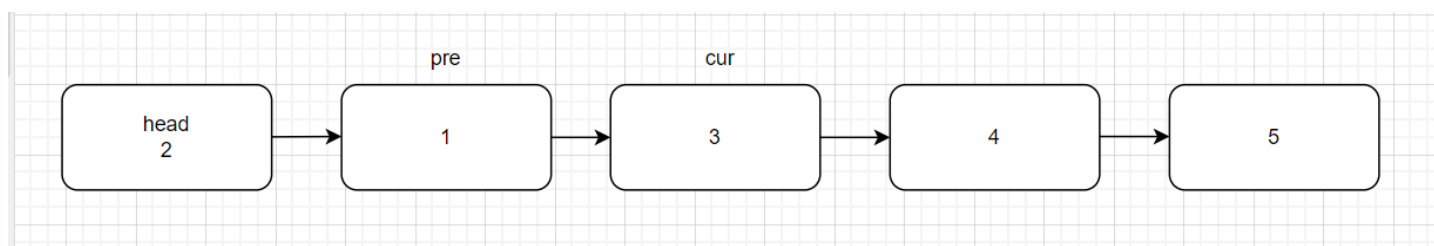
(3) 接: **head = cur**

更新head, 使其指向当前的cur



(4) 移: **cur = pre->next**

更新cur, 移动到下一个待反转的节点



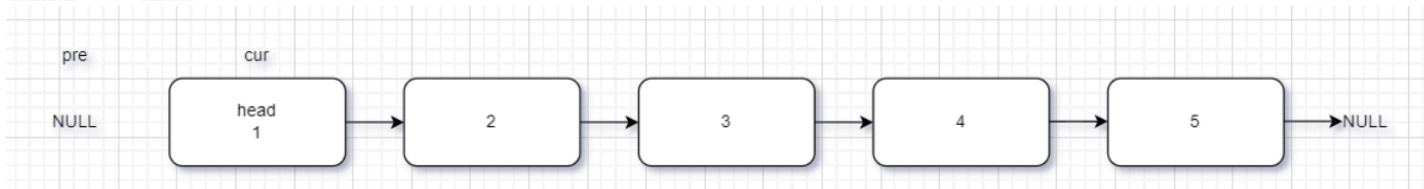
原地翻转

```

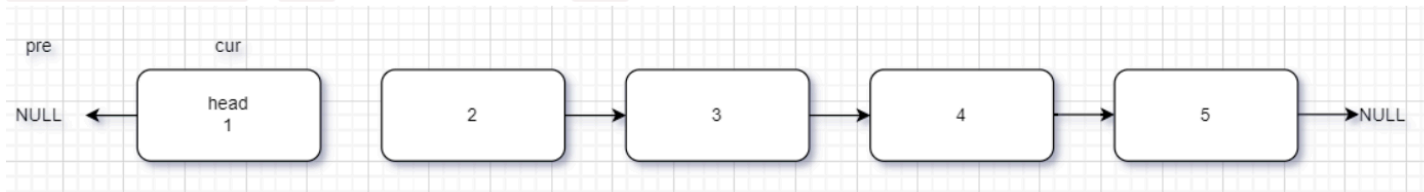
1  struct ListNode* reverseList(struct ListNode* head) {
2      if (head == NULL) {
3          return head;
4      }
5      struct ListNode* cur = head->next;
6      struct ListNode* pre = head;
7      while (cur) {
8          pre->next = cur->next;
9          cur->next = head;
10         head = cur;
11         cur = pre->next;
12     }
13     return head;
14 }
  
```

2.迭代

temp 保存 cur 的下一个节点，以防止反转时丢失链表信息。

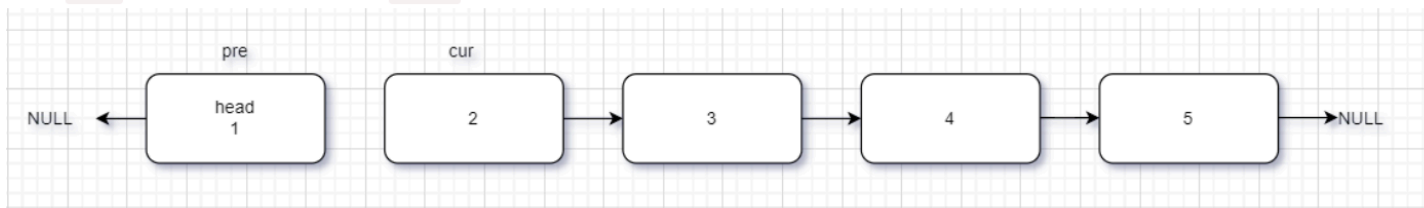


cur->next = pre; 将 cur 的下一个指针指向 pre, 实现反转。



更新 pre 为当前的 cur, 为下一次迭代做准备。

更新 cur 为保存的下一个节点 temp, 继续迭代。



通过不断改变指针的指向将链表进行反转。pre 充当新链表的头节点, 当 cur 为 NULL 循环停止, 返回 pre

迭代

```
1  struct ListNode* reverseList(struct ListNode* head) {
2      struct ListNode* pre = NULL;
3      struct ListNode* cur = head;
4      while (cur != NULL) {
5          struct ListNode* temp = cur->next;
6          cur->next = pre;
7          pre = cur;
8          cur = temp;
9      }
10     return pre;
11 }
```

3.递归

我们可以通过迭代的思路来得到递归方法

观察reverse函数：

函数声明中 `pre` 指针指向的为 `NULL`，`cur` 指针指向的为 `head`，正如迭代中声明并初始化 `pre` 和 `cur` 指针
递归结束条件为 `cur` 为 `NULL`，返回 `pre`

同样 `temp` 保存 `cur` 的下一个节点，以防止反转时丢失链表信息。

然后进行反转 `cur->next = pre;`

`pre` 为当前已反转部分的头节点，`cur` 为当前待反转的节点。

然后调用递归，将 `cur` 作为新的 `pre` 传入下一层，将 `temp` 作为新的 `cur` 传入下一层。

这样就实现了链表的递归反转

递归

```
1  struct ListNode* reverse(struct ListNode* pre, struct ListNode* cur) {
2      if (!cur){
3          return pre;
4      }
5      struct ListNode* temp = cur->next;
6      cur->next = pre;
7      //将cur作为pre传入下一层
8      //将temp作为cur传入下一层，改变其指针指向当前cur
9      return reverse(cur, temp);
10 }
11
12 struct ListNode* reverseList(struct ListNode* head) {
13     return reverse(NULL, head);
14 }
```

4.新建链表

遍历原链表，使用头插法新建链表，并不断移动 `newHead` 指针，当原链表头指针指向 `NULL` 最终返回 `newHead`

新建链表

```
1
2  struct ListNode* reverseList(struct ListNode* head) {
3      struct ListNode* newHead = NULL;
4      while (head) {
5          // 创建一个新节点
6          struct ListNode* newNode = (struct ListNode*)malloc(sizeof(struct
7          ListNode));
8          newNode->val = head->val;
9          // 将新节点插入到新链表的头部
10         newNode->next = newHead;
11         newHead = newNode;
12         // 移动到原链表的下一个节点
13         head = head->next;
14     }
```

```
13     }  
14     return newHead;  
15 }
```